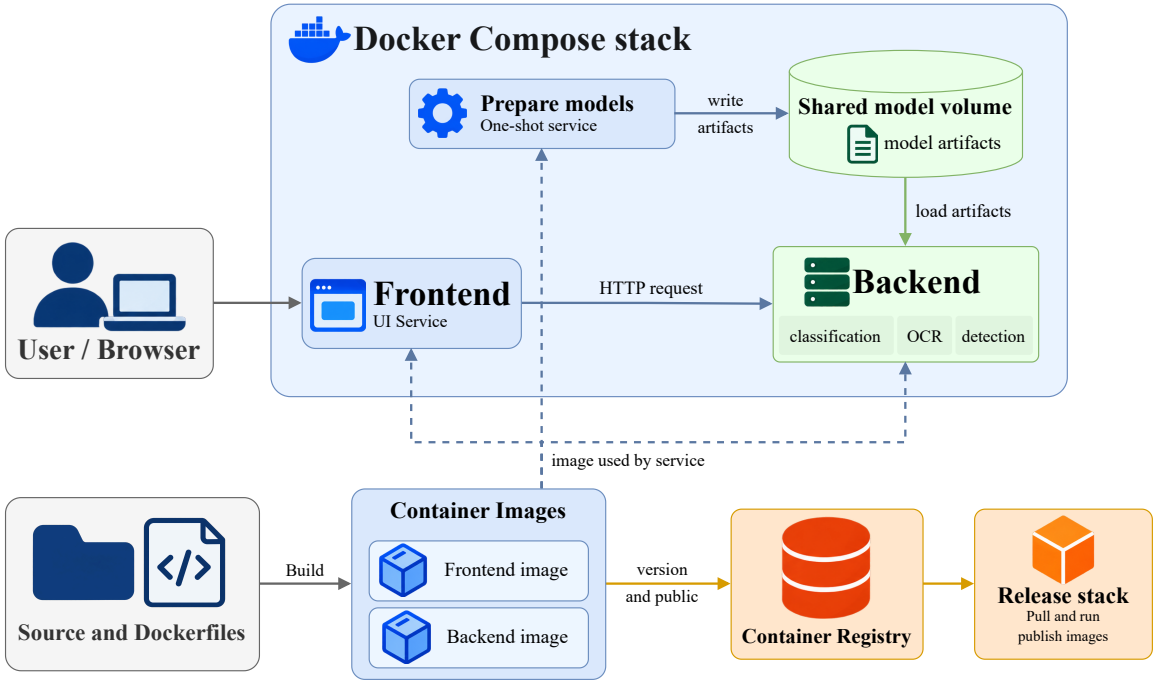
Packaging AI Inference with Docker (Cơ bản)

Trần Quang Minh **Nguyễn Đăng Nhã** **Đinh Quang Vinh**

I. Giới thiệu

Ứng dụng AI có thể hoạt động tốt trên máy dùng để phát triển nhưng gặp lỗi khi chuyển sang máy khác. Ngoài source code, ứng dụng còn phụ thuộc vào thư viện, model files, cấu hình và cách khởi động. Khi dự án có thêm UI, server và nhiều model, việc chuẩn bị từng thành phần bằng tay trở nên khó quản lý. Docker giúp package môi trường chạy và tổ chức quá trình chạy ứng dụng theo cách nhất quán.



Hình 1: Tổng quan kiến trúc và workflow Docker của AI application.

Bài viết bắt đầu từ việc tạo image và chạy ứng dụng trong container. Tiếp theo là cách quản lý quá trình build, lưu model artifacts và vận hành các thành phần bằng Docker Compose. Phần cuối trình bày cách tối ưu image, đưa image lên container registry và chạy bản release.

Mục lục

I. Giới thiệu **1**

II. Containerize ứng dụng **3**

II.1. Xây dựng image cho ứng dụng 3

II.2. Tạo và chạy container 4

II.3. Kiểm tra hoạt động của service trong container 6

III. Quản lý môi trường chạy và quá trình build **6**

III.1. Runtime và dependencies 7

III.2. Image layers và build cache 8

III.3. Versioning và kiểm tra image 9

IV. Quản lý model artifacts và cấu hình **11**

IV.1. Tổ chức model artifacts và cấu hình khi chạy 11

IV.2. Chuẩn bị model artifacts trên volume và thực hiện inference 12

V. Vận hành ứng dụng bằng Docker Compose **17**

V.1. Tổ chức các thành phần của ứng dụng 18

V.2. Kết nối các service và xác định thứ tự khởi động 21

V.3. Kiểm tra cấu hình và vận hành ứng dụng 24

VI. Tối ưu và publish Docker image **28**

VI.1. Kiểm tra các chức năng chính của ứng dụng 28

VI.2. Tối ưu Docker image 29

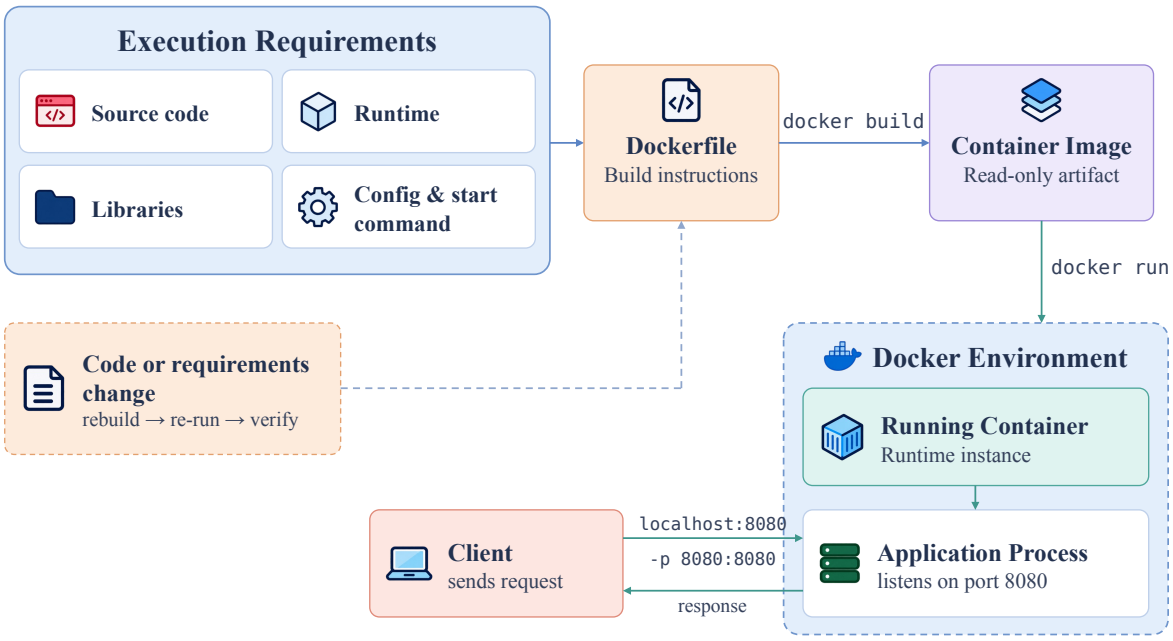
VI.3. Đưa image lên registry và chạy bản release 32

Phụ lục **35**

II. Containerize ứng dụng

Để chạy được trên một môi trường, ứng dụng không chỉ cần source code mà còn phụ thuộc vào runtime phù hợp, các thư viện cần thiết, cấu hình và lệnh khởi động. Nếu những thành phần này khác nhau giữa các môi trường, ứng dụng có thể không khởi động được hoặc không hoạt động như mong đợi. Containerization giúp chuẩn hóa cách chuẩn bị runtime environment, nhờ đó ứng dụng có thể được deploy nhất quán hơn ở nơi khác.

Để tạo ra môi trường đó, Docker sử dụng các chỉ dẫn trong Dockerfile để build một container image. Image là một artifact chỉ đọc, được dùng làm khuôn mẫu để tạo container. Khi container được khởi chạy, ứng dụng bắt đầu thực thi bên trong môi trường này. Ta có thể gửi request đến endpoint của service qua port đã được publish, rồi kiểm tra response để xác nhận service hoạt động đúng.



Hình 2: Quy trình tổng quát khi containerize một ứng dụng, từ runtime requirements đến image, container đang chạy và bước kiểm tra service.

Project này sử dụng một backend FastAPI để minh họa quy trình trên, nhưng các backend framework khác cũng đi qua những bước tương tự. Khi source code hoặc runtime requirements thay đổi, image được build lại, một container mới được khởi chạy và service được kiểm tra lại.

II.1. Xây dựng image cho ứng dụng

Các thành phần cần có trong image được mô tả bằng những chỉ dẫn trong Dockerfile, nhờ đó Docker biết cách thiết lập môi trường và đưa các file cần thiết vào image. Với một ứng dụng Python, Dockerfile có thể được tổ chức như sau:

Dockerfile: ví dụ cho một ứng dụng Python

```
FROM python:3.11-slim

WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY . .

CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8080"]
```

Mỗi chỉ dẫn trong ví dụ trên phụ trách một phần của quá trình chuẩn bị image:

- `FROM` chọn base image `python:3.11-slim`, trong đó Python runtime đã được chuẩn bị sẵn.
- `WORKDIR` đặt `/app` làm thư mục làm việc bên trong image.
- `COPY requirements.txt .` đưa file liệt kê các thư viện cần cài đặt vào thư mục làm việc.
- `RUN pip install --no-cache-dir -r requirements.txt` cài các thư viện được liệt kê trong `requirements.txt`.
- `COPY . .` đưa các file của ứng dụng vào image.
- `CMD [...]` khai báo lệnh mặc định dùng để khởi động ứng dụng khi container chạy.

Trong chuỗi chỉ dẫn này, Docker dùng `RUN` và `CMD` ở hai thời điểm khác nhau. `RUN` được thực thi trong quá trình build và những thay đổi do lệnh tạo ra được lưu vào image, vì vậy các thư viện Python trong ví dụ được cài đặt ở giai đoạn này. `CMD` không được thực thi khi build mà chỉ khai báo lệnh mặc định để Docker sử dụng khi container được khởi chạy từ image.

Khi quá trình build hoàn tất, Docker tổ chức kết quả của các bước trên thành những layer chỉ đọc, từ đó tạo nên container image. Dockerfile vì thế đóng vai trò như công thức, còn image là artifact được tạo ra từ công thức đó. Ở bước tiếp theo, Docker sử dụng image làm khuôn mẫu để tạo container.

II.2. Tạo và chạy container

Container image được tạo ở bước trước vẫn là một artifact chỉ đọc. Khi thực hiện `docker run`, Docker dùng các layer của image làm nền để tạo container, sau đó khởi động process của ứng dụng theo command đã được cấu hình trong image. Vì các image layer ban đầu không bị thay đổi, cùng một image có thể được dùng để tạo nhiều container độc lập, mỗi container duy trì trạng thái riêng trong quá trình chạy.

Container

Một runtime instance được tạo từ container image để thực thi application process.

Để lưu những thay đổi phát sinh trong filesystem, mỗi container có một **writable layer** riêng phía trên các image layer chỉ đọc. Các thay đổi trong lúc container chạy được ghi vào layer này thay vì sửa trực tiếp image gốc. Vì writable layer thuộc riêng từng container, thay đổi trong một container không làm ảnh hưởng đến image hoặc trạng thái filesystem của các container khác.

Writable layer

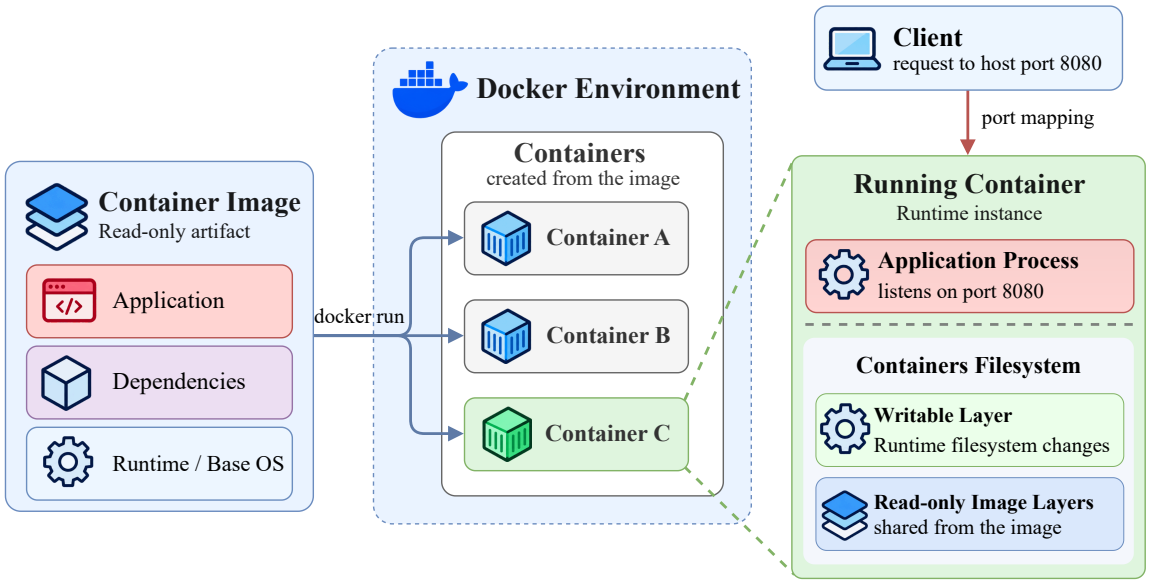
Layer có thể ghi được gắn riêng với mỗi container để lưu những thay đổi phát sinh trong filesystem khi container đang chạy.

Nếu ứng dụng cung cấp service qua mạng, process bên trong container sẽ lắng nghe trên một port, chẳng hạn `8080`. Việc process lắng nghe trên port này chưa tự động cho phép client bên ngoài truy cập. Khi chạy container, Docker có thể **publish** một port trên host và map nó tới port mà ứng dụng đang lắng nghe bên trong container. Với tùy chọn `-p 8080:8080`, số đầu tiên là port trên host và số thứ hai là port bên trong container. Vì vậy, request gửi tới port `8080` trên host sẽ được chuyển tới ứng dụng đang lắng nghe tại port `8080` trong container. Hai port không bắt buộc phải giống nhau, nên `-p 8081:8080` vẫn giữ ứng dụng ở port `8080` nhưng cho phép client truy cập qua port `8081` trên host.

Port publishing

Cơ chế map một port trên host tới port mà ứng dụng đang lắng nghe bên trong container, qua đó cho phép client truy cập service thông qua host.

Chỉ dẫn `EXPOSE 8080` trong Dockerfile chỉ mô tả port mà ứng dụng dự kiến sử dụng bên trong container. Nó không tự publish port ra host, vì vậy khi chạy container vẫn cần sử dụng tùy chọn `-p` nếu muốn client bên ngoài truy cập service.



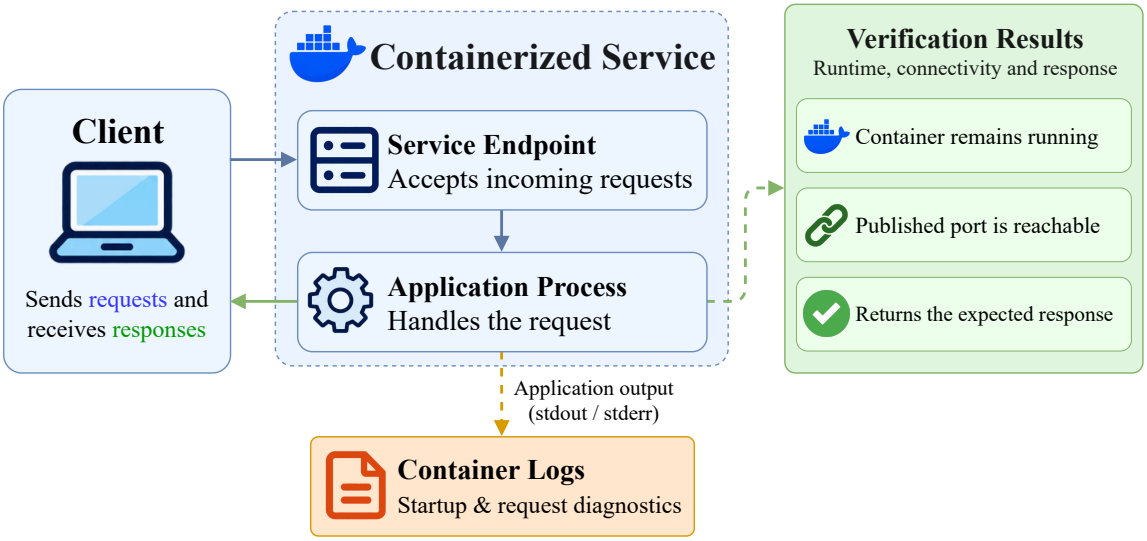
Hình 3: Quan hệ giữa image, running container và đường đi của request tới ứng dụng trong container.

Sau khi image được khởi chạy thành container và port được publish, service đã sẵn sàng cho bước kiểm tra tiếp theo.

II.3. Kiểm tra hoạt động của service trong container

Sau khi container được khởi chạy và port được publish, client có thể gửi request tới endpoint để kiểm tra đường truy cập từ bên ngoài vào service. Process của ứng dụng tiếp nhận request tại endpoint, thực hiện phần xử lý tương ứng và trả response về cho client. Nếu response có cấu trúc như dự kiến, ta có thể xác nhận request đã đi qua port được publish và service xử lý được request tại endpoint đã cấu hình. Để phép kiểm tra này không phụ thuộc vào một dependency chưa sẵn sàng, dependency đó có thể được thay bằng mock nhằm tạo ra response có thể dự đoán. Trong project hiện tại, model inference được mock để có thể xác minh backend một cách độc lập trước khi model artifact thật được tích hợp.

Bước xác minh này kiểm tra khả năng truy cập service và luồng xử lý request ở mức cơ bản, nhưng chưa thay thế cho việc kiểm tra đầy đủ logic của ứng dụng. Nếu service không khởi động hoặc request không được xử lý như mong đợi, có thể xem output mà ứng dụng ghi vào `stdout` và `stderr` bằng `docker logs <container_name>`. Các log này cung cấp thông tin trong quá trình khởi động và xử lý request, qua đó hỗ trợ tìm nguyên nhân của lỗi bên trong container.



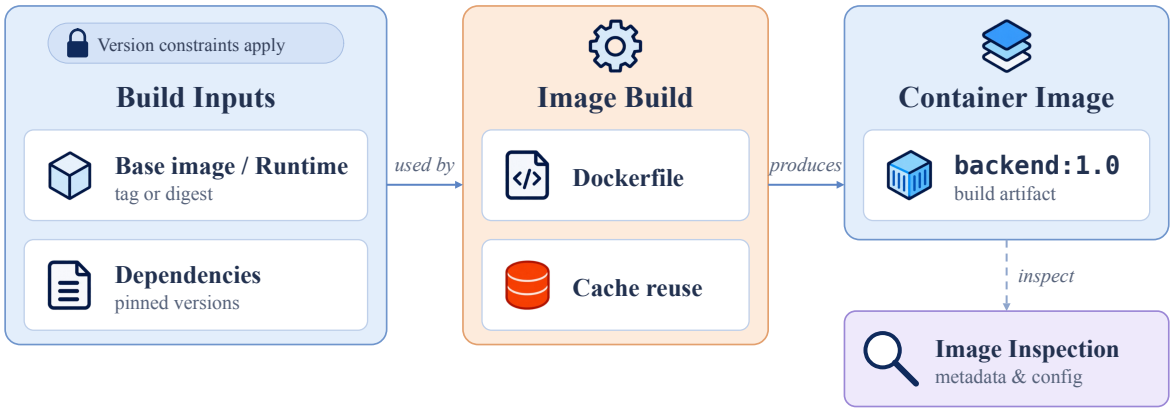
Hình 4: Luồng xác minh containerized service thông qua request, response và container logs.

Qua bước này, backend đã đi qua quy trình containerization cơ bản từ Dockerfile và image đến running container, sau đó được truy cập và kiểm tra thông qua service endpoint. Quy trình tương tự có thể được lặp lại khi source code hoặc runtime requirements của ứng dụng thay đổi.

III. Quản lý môi trường chạy và quá trình build

Ở mục trước, Dockerfile được sử dụng để tạo image, sau đó image được khởi chạy thành container và service được kiểm tra từ bên ngoài. Khi image phải được build lại mỗi lần ứng dụng thay đổi, runtime và dependencies trở thành những build input cần kiểm soát version, trong khi cách

tổ chức các instruction trong Dockerfile ảnh hưởng đến khả năng tái sử dụng build cache. Sau quá trình build, container image được tạo ra như một artifact và có thể được kiểm tra lại thông qua configuration và metadata. Vì vậy, mục này tập trung vào ba khía cạnh: xác định runtime environment, tổ chức quá trình build hiệu quả và kiểm tra image được tạo ra.



Hình 5: Mối quan hệ giữa các build input, quá trình Docker xử lý Dockerfile, container image được tạo ra và thao tác kiểm tra image sau khi build.

III.1. Runtime và dependencies

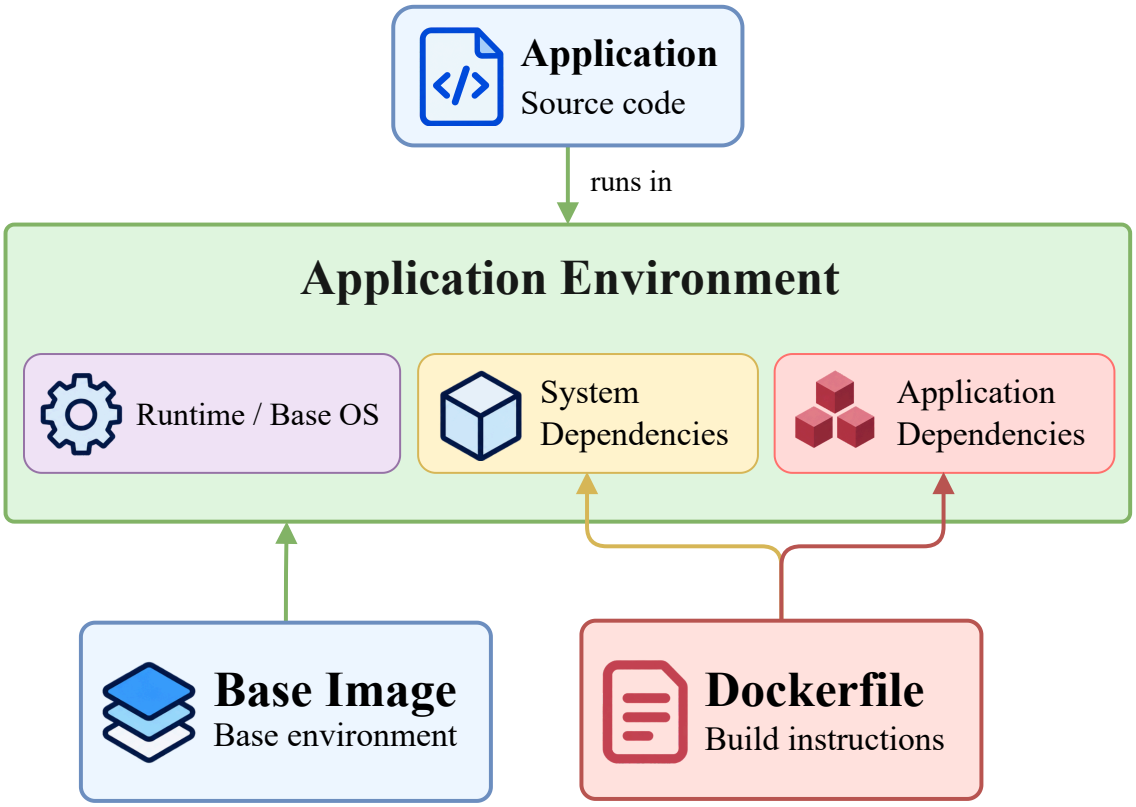
Để source code có thể được thực thi bên trong container, image cần cung cấp runtime phù hợp cùng các dependencies mà ứng dụng sử dụng. Với backend Python trong project hiện tại, Python đóng vai trò runtime, còn những thư viện và thành phần hệ thống chưa có sẵn sẽ được bổ sung trong quá trình build.

Trong Docker, runtime environment thường bắt đầu từ một **base image** được khai báo bằng `FROM`. Chẳng hạn, `python:3.11-slim` cung cấp môi trường nền đã có Python 3.11 để Docker tiếp tục thiết lập những thành phần mà ứng dụng cần. Runtime nhờ đó có thể được kế thừa từ base image thay vì phải thiết lập lại từ đầu trong Dockerfile.

Các dependencies có thể được chia thành hai nhóm dựa trên vai trò của chúng:

- **System dependencies** gồm các thư viện hệ điều hành hoặc công cụ mà ứng dụng hay quá trình cài đặt cần sử dụng.
- **Application dependencies** là các thư viện và package phục vụ trực tiếp cho ứng dụng. Với Python backend, các package này thường được khai báo trong `requirements.txt` và cài vào image trong quá trình build.

Base image cung cấp nền ban đầu cho môi trường, còn các instruction trong Dockerfile xác định những dependencies nào cần bổ sung. Khi runtime và dependencies đã được xác định, cách sắp xếp các instruction này sẽ ảnh hưởng đến khả năng tái sử dụng build cache khi image được build lại.



Hình 6: Quan hệ giữa base image, runtime và các nhóm dependencies tạo nên runtime environment của ứng dụng.

III.2. Image layers và build cache

Khi Docker build một image, các instruction trong Dockerfile được xử lý theo thứ tự từ trên xuống. Kết quả của nhiều bước build được tổ chức thành các **layer** chỉ đọc, sau đó các layer này được xếp chồng để hình thành image. Cấu trúc theo layer cũng là nền tảng để Docker tái sử dụng kết quả của những bước đã được thực hiện trước đó.

Image layer

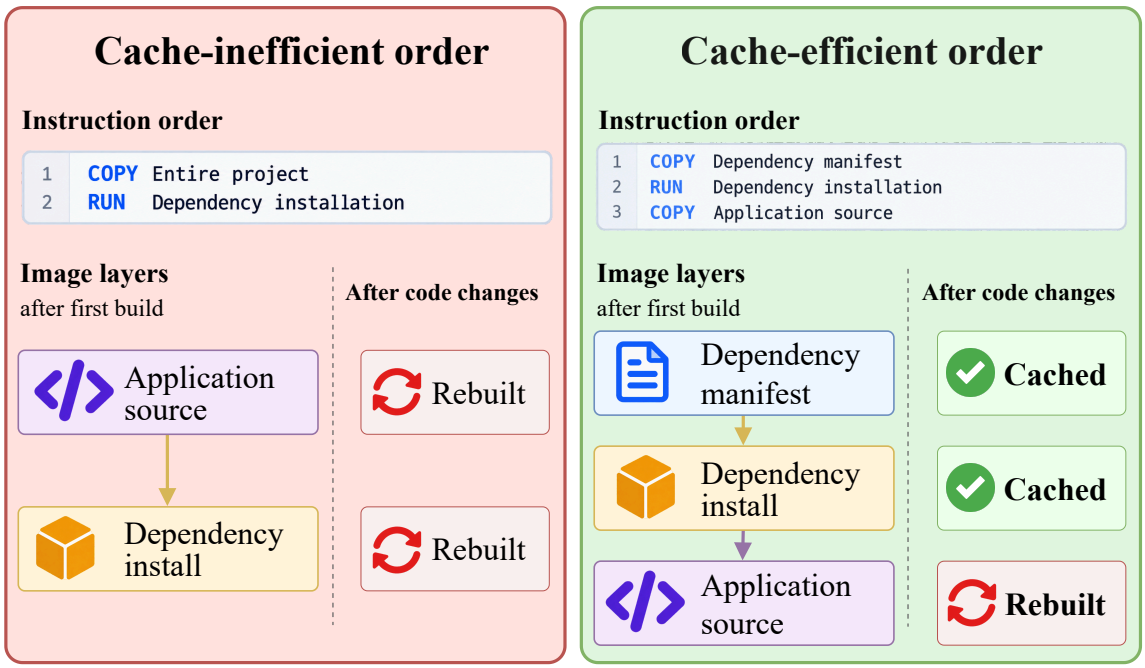
Một layer chỉ đọc chứa những thay đổi do một bước build tạo ra và được xếp chồng lên các layer trước đó để hình thành image.

Trong lần build tiếp theo, Docker kiểm tra từng bước để xác định kết quả tương ứng trong **build cache** còn có thể được sử dụng hay không. Nếu instruction và các input liên quan không thay đổi, Docker tái sử dụng kết quả trước đó thay vì thực hiện lại bước này. Khi một bước không còn khớp với cache, Docker phải thực hiện lại bước đó cùng các bước theo sau trong cùng chuỗi build.

Build cache

Kết quả của các bước build trước đó được Docker lưu lại để tái sử dụng khi instruction và các input liên quan không thay đổi.

Để hạn chế số bước phải build lại, các instruction thường được sắp xếp theo nguyên tắc **stable first, volatile last**, tức những thành phần ít thay đổi được xử lý trước và những thành phần thay đổi thường xuyên hơn được đưa về sau. Chẳng hạn, file khai báo dependencies thường ổn định hơn source code ứng dụng. Nếu file này được copy và dependencies được cài đặt trước khi source code được đưa vào image, một thay đổi chỉ ở source code vẫn có thể giữ lại cache của bước cài dependencies. Ngược lại, nếu toàn bộ project được copy trước, thay đổi trong source code sẽ khiến bước `COPY` và các bước phía sau không thể tái sử dụng cache, ngay cả khi danh sách dependencies không đổi.



Hình 7: Ảnh hưởng của thứ tự các instruction trong Dockerfile đến image layers và khả năng tái sử dụng build cache khi source code thay đổi.

III.3. Versioning và kiểm tra image

Dùng cùng một Dockerfile không có nghĩa là mọi lần build đều sử dụng đúng những thành phần như nhau. Base image có thể được cập nhật theo thời gian, hoặc một dependency có thể release version mới. Khi đó, lần build sau có thể lấy về những thành phần khác dù source code và Dockerfile vẫn giữ nguyên.

Để hạn chế tình huống này, cần kiểm soát hai nhóm input:

- **Base image** nên được tham chiếu đủ cụ thể. Một tag như `3.11-slim` giúp chọn rõ version hoặc biến thể hơn so với `latest`, nhưng tag vẫn có thể được cập nhật để trỏ tới nội dung

image khác. Nếu cần cố định chính xác nội dung của base image, có thể sử dụng **digest** (định danh gắn với nội dung của image).

- **Dependencies** nên được **pin version** (chỉ định một version cụ thể) thay vì để công cụ cài đặt tự chọn một version khác ở lần build sau. Nhờ đó, các package được đưa vào environment được kiểm soát rõ hơn.

Những constraint này giúp hạn chế thay đổi ngoài dự kiến ở input của quá trình build, nhưng chưa bảo đảm toàn bộ build luôn tạo ra kết quả hoàn toàn giống nhau.

Sau khi build, image cũng cần có cách nhận diện để chạy, kiểm tra hoặc sử dụng lại. Docker sử dụng **image reference** cho mục đích này. Chẳng hạn, trong `backend:1.0`, `backend` là repository còn `1.0` là tag dùng để phân biệt version hoặc biến thể của image trong repository đó.

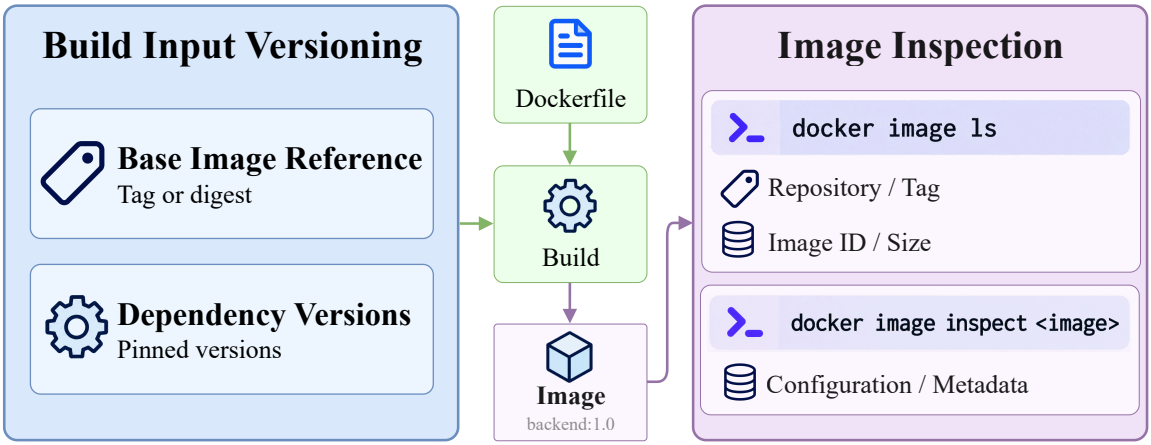
Image reference

Chuỗi dùng để tham chiếu một container image, thường gồm repository kết hợp với tag hoặc digest.

Build thành công chỉ cho biết Docker đã tạo được image. Khi cần xem các image hiện có hoặc kiểm tra thông tin chi tiết của một image cụ thể, có thể sử dụng:

- `docker image ls` để xem repository, tag, image ID và size của các image hiện có. Có thể thêm `--digests` khi cần hiển thị digest.
- `docker image inspect <image>` để xem **configuration** (cấu hình của image) và **metadata** (các thông tin mô tả image) chi tiết hơn.

Các command này giúp kiểm tra image sau build, chẳng hạn image đang được tham chiếu bằng tag nào và có configuration như thế nào. Còn việc service có hoạt động đúng hay không thì vẫn phải kiểm tra khi chạy image thành container.



Hình 8: Mối quan hệ giữa việc kiểm soát version của các build input, quá trình tạo image và việc kiểm tra image sau build.

IV. Quản lý model artifacts và cấu hình

Backend hiện đã được package thành container image, nhưng để chạy inference, ứng dụng vẫn cần model weights, classifier và các file liên quan. Nếu đưa toàn bộ dữ liệu này vào image, mỗi lần cập nhật model đều có thể phải build lại image dù application code không thay đổi. Vì vậy, phần này sẽ tách application image, model artifacts và runtime configuration để quản lý mỗi thành phần theo đúng vai trò của nó.

IV.1. Tổ chức model artifacts và cấu hình khi chạy

Từ mục này trở đi, tutorial sử dụng trực tiếp repo mẫu được cung cấp trong **Phụ lục**. Trước khi lựa chọn cách lưu model artifacts, ta cần xác định backend image hiện chứa những gì và ứng dụng còn cần thêm những thành phần nào để chạy inference.

Cấu trúc thư mục: backend/ và frontend/

```
backend/
|-- Dockerfile
|-- requirements.txt
|-- pyproject.toml
|-- uv.lock
|-- scripts/
|   `-- prepare_models.py
|-- model-data/
`-- app/
    |-- main.py
    |-- core/
    |   `-- config.py
    |-- models/
    |-- services/
    `-- routers/

frontend/
|-- app.py
`-- requirements.txt

.env.example
```

Khi build, Docker xử lý các phần trong cấu trúc trên theo những cách khác nhau:

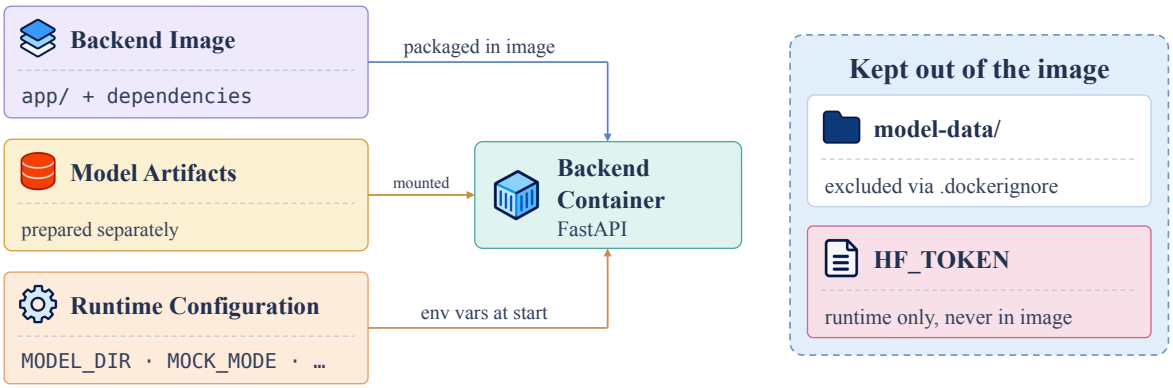
Thành phần	Cách được xử lý
`app/`	Được copy vào backend image
Python dependencies	Được cài đặt từ `requirements.txt`
`scripts/`	Được copy vào backend image
`model-data/`	Bị `.dockerignore` loại khỏi dữ liệu gửi cho Docker khi build

Backend image vì vậy chứa application code, Python dependencies và các script hỗ trợ, nhưng không chứa những file cần cho inference như model weights, file cấu hình và classifier. Trong

phần này, các file đó được gọi chung là **model artifacts**. Repo hiện cần model artifacts cho RapidOCR, TrOCR, YOLOS tiny và Iris classifier.

Từ đó, các thành phần mà backend cần có thể được chia thành ba nhóm:

- Application code và Python dependencies được package trong backend image.
- Model artifacts được chuẩn bị và lưu riêng.
- Các giá trị cấu hình được truyền vào khi container chạy.



Hình 9: Phân tách application image, model artifacts và runtime configuration trong backend.

Repo đã có `scripts/prepare_models.py` để chuẩn bị model artifacts vào thư mục do `MODEL_DIR` xác định. Khi chạy local, thư mục mặc định là `backend/model-data/`, còn trong container là `/models`. Những thiết lập thay đổi theo môi trường được truyền vào container qua các biến môi trường:

- `MODEL_DIR` xác định vị trí model artifacts.
- `INFERENCE_DEVICE` xác định device dùng cho inference.
- `MOCK_MODE` điều khiển chế độ mock của các service có hỗ trợ.
- `HF_HUB_OFFLINE` ngăn việc truy cập Hugging Face Hub trong runtime.

Các biến trên được mô tả bằng giá trị mẫu trong `.env.example`. Riêng `HF_TOKEN` chỉ được truyền vào khi bước chuẩn bị model cần xác thực với Hugging Face Hub, nhờ đó token không bị lưu trong Dockerfile hoặc image và không bị lộ cho những người có quyền truy cập image.

Sau khi tách application image, model artifacts và runtime configuration theo các vai trò trên, phần tiếp theo sẽ xác định nơi lưu model artifacts để container có thể sử dụng chúng qua nhiều lần chạy.

IV.2. Chuẩn bị model artifacts trên volume và thực hiện inference

Khi model artifacts không còn được package trực tiếp vào backend image, ứng dụng cần một nơi lưu trữ riêng cho các file cần cho inference. Docker cung cấp volume, một vùng lưu trữ được tách khỏi filesystem riêng của container. Nhờ đó, dữ liệu có thể tiếp tục tồn tại ngay cả khi container sử dụng nó bị xóa.

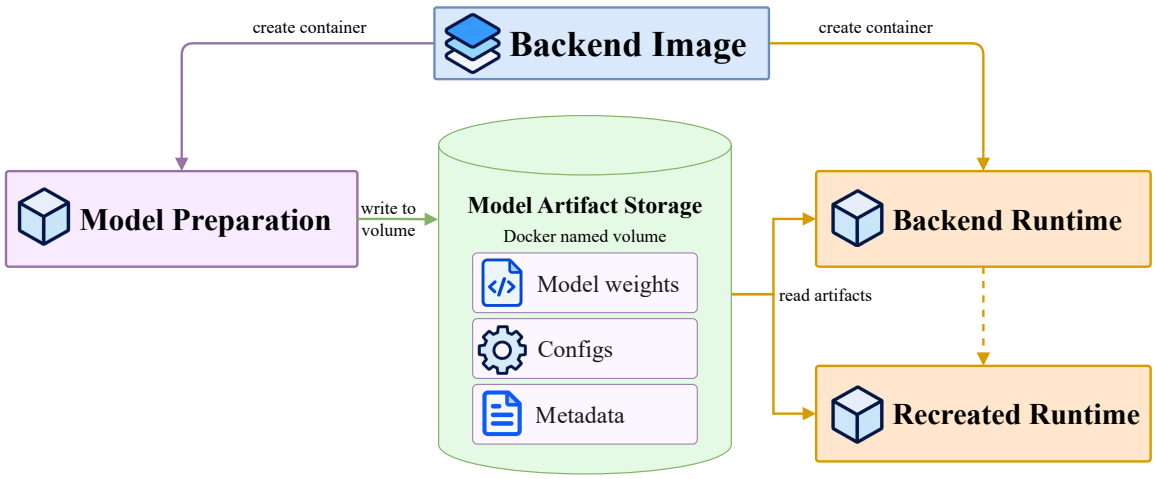
Volume

Là vùng lưu trữ được tách khỏi filesystem của container và có thể tồn tại độc lập với vòng đời của container.

Tutorial này sử dụng một named volume tên `model-artifacts`. Đây là volume có tên do Docker quản lý, nhờ đó các container khác nhau có thể mount và sử dụng lại cùng dữ liệu. Một lựa chọn khác là bind mount, cơ chế map trực tiếp một file hoặc thư mục trên host vào container. Bind mount phù hợp khi container cần làm việc trực tiếp với dữ liệu do người dùng quản lý trên host, còn model artifacts trong workflow này phù hợp hơn với named volume vì chúng không cần gắn với một đường dẫn cụ thể trong repository.

Named volume

Là volume có tên do Docker quản lý và có thể được sử dụng lại bởi nhiều container.



Hình 10: Chuẩn bị và tái sử dụng model artifacts qua Docker named volume.

Trước khi bắt đầu, build backend image từ repository root:

PowerShell: build backend image

```
docker build -t backend:1.0 ./backend
```

Bước 1. Tạo volume

Tạo volume dùng để lưu model artifacts:

PowerShell: tạo volume model-artifacts

```
docker volume create model-artifacts
```

Lệnh này chỉ bảo đảm `model-artifacts` tồn tại. Nếu volume cùng tên đã có từ trước, Docker sẽ sử dụng lại volume đó thay vì tạo một volume trống. Khi muốn chạy lại workflow từ trạng thái sạch, có thể xóa backend container và volume cũ trước khi tạo lại. Việc xóa volume cũng xóa toàn bộ model artifacts đã lưu bên trong, vì vậy thao tác này chỉ nên thực hiện khi thực sự muốn bắt đầu lại từ đầu.

PowerShell: xóa container, volume cũ và tạo lại volume

```
docker rm -f ai-backend 2>$null
docker volume rm model-artifacts 2>$null
docker volume create model-artifacts
```

Trong những lần chạy thông thường, `model-artifacts` được giữ lại để backend tiếp tục sử dụng các model đã chuẩn bị mà không phải tải lại.

Bước 2. Chuẩn bị model artifacts

Thay vì để backend tự tải model khi khởi động, ta dùng chính image `backend:1.0` để chạy `prepare_models.py`. Trong tutorial này, container tạm thời làm nhiệm vụ này được gọi là **preparation container**. Nó chỉ tồn tại trong thời gian chuẩn bị model, ghi dữ liệu vào volume rồi kết thúc khi script hoàn thành.

Preparation container mount `model-artifacts` vào `/models` với quyền đọc và ghi. Biến `MODEL_DIR=/models` cho script biết nơi cần lưu model weights, classifier, metadata và các file khác cần cho inference.

PowerShell: chạy preparation container

```
docker run --rm `
  --name prepare-models `
  --mount type=volume,source=model-artifacts,target=/models `
  -e MODEL_DIR=/models `
  backend:1.0 `
  python scripts/prepare_models.py
```

Preparation container cần kết nối Internet để tải các Hugging Face models. Với model public, không cần `HF_TOKEN`. Token chỉ cần được truyền vào khi model repository yêu cầu xác thực.

Trong lệnh trên, `source=model-artifacts` chọn volume do Docker quản lý, còn `target=/models` xác định vị trí volume xuất hiện trong filesystem của container. Application không cần biết Docker lưu volume ở đâu trên host mà chỉ làm việc với đường dẫn `/models` được cung cấp qua `MODEL_DIR`.

Khi backend chạy trực tiếp ngoài Docker, project vẫn có thể sử dụng một đường dẫn local như `backend/model-data`. Trong container, application sử dụng `/models`. Nhờ `MODEL_DIR`, code không cần gắn cố định với vị trí lưu model ở từng môi trường.

Sau khi preparation hoàn tất, có thể liệt kê các artifacts đã được tạo trong volume. `sort` giúp thứ tự output ổn định hơn giữa các lần chạy, còn `-maxdepth 2` tránh đi quá sâu vào những thư mục cache.

PowerShell: liệt kê model artifacts trong volume

```
docker run --rm `
  --mount type=volume,source=model-artifacts,target=/models,readonly `
  backend:1.0 `
  sh -c "find /models -mindepth 1 -maxdepth 2 -type f | sort"
```

Ở bước này, volume được mount ở chế độ read-only vì container chỉ cần đọc dữ liệu. Kết quả chỉ cho thấy các file đã xuất hiện trong `model-artifacts`, chưa chứng minh rằng model có thể được load và thực hiện inference thành công.

Bước 3. Chạy backend với model đã chuẩn bị

Sau khi artifacts đã được chuẩn bị, backend mount lại cùng `model-artifacts` tại `/models`. Lần này volume được đặt ở chế độ read-only vì backend chỉ cần sử dụng dữ liệu đã có. Runtime cũng bật `HF_HUB_OFFLINE=1`, tức Hugging Face offline mode, để application chỉ sử dụng artifacts đã được chuẩn bị thay vì tải thêm dữ liệu trong lúc inference.

PowerShell: chạy backend container

```
docker run -d `
  --name ai-backend `
  -p 8000:8000 `
  --mount type=volume,source=model-artifacts,target=/models,readonly `
  -e MODEL_DIR=/models `
  -e INFERENCE_DEVICE=cpu `
  -e MOCK_MODE=0 `
  -e HF_HUB_OFFLINE=1 `
  backend:1.0
```

`MODEL_DIR`, `INFERENCE_DEVICE`, `MOCK_MODE` và `HF_HUB_OFFLINE` là các runtime configuration. Những giá trị này được cung cấp khi container được tạo, vì vậy có thể thay đổi cách backend chạy mà không cần build lại image. Danh sách model và revision mà application hỗ trợ vẫn được xác định trong code.

Do `docker run -d` trả lại terminal ngay trong khi Uvicorn có thể chưa sẵn sàng nhận request, nên cần theo dõi log của container trước khi kiểm tra API:

PowerShell: theo dõi log của backend

```
docker logs -f ai-backend
```

Chờ đến khi log xuất hiện `Application startup complete`, sau đó nhấn `Ctrl+C` để dừng việc theo dõi log. Thao tác này không dừng backend container. Khi backend đã sẵn sàng, trước tiên kiểm tra `/health` để xác nhận API đang phản hồi với cấu hình runtime mong muốn:

PowerShell: kiểm tra /health

```
Invoke-RestMethod http://localhost:8000/health
```

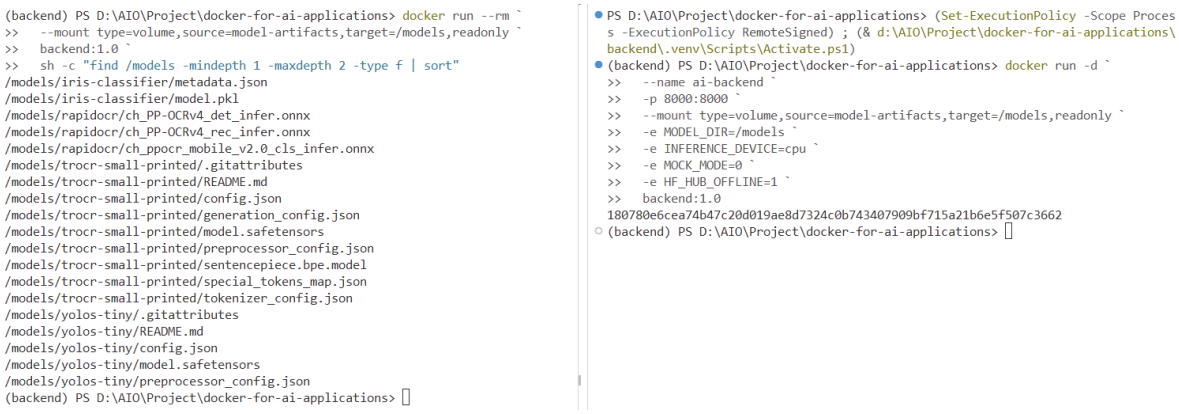
Sau đó gửi một request tới Iris classification endpoint để kiểm tra inference thật:

PowerShell: gửi request Iris classification

```
$body = @{
  sepal_length = 5.1
  sepal_width = 3.5
  petal_length = 1.4
  petal_width = 0.2
} | ConvertTo-Json

Invoke-RestMethod `
  -Uri http://localhost:8000/classification/predict `
  -Method Post `
  -ContentType "application/json" `
  -Body $body
```

Iris classification xác nhận backend có thể load một model artifact từ volume và thực hiện prediction thật trên CPU. Các endpoint inference còn lại như RapidOCR, TrOCR và YOLOS sẽ được kiểm tra đầy đủ hơn bằng smoke test ở phần kiểm chứng ứng dụng.



Hình 11: Kết quả kiểm tra model artifacts và runtime inference.

Bước 4. Kiểm tra dữ liệu sau khi thay thế container

Model artifacts được lưu trong `model-artifacts`, thay vì writable layer của backend container. Volume được mount vào `/models` khi backend chạy. Để kiểm tra volume thực sự có vòng đời riêng, xóa container đang chạy nhưng giữ nguyên volume:

PowerShell: xóa backend container, giữ lại volume

```
docker rm -f ai-backend
```

Sau đó chạy lại command tạo backend ở Bước 3 mà không chạy lại `prepare_models.py`, rồi gửi lại cùng request Iris classification. Nếu prediction tiếp tục hoạt động, backend mới đã sử dụng lại những artifacts được chuẩn bị từ trước. Điều này cho thấy backend container có thể bị xóa và tạo lại trong khi model artifacts vẫn được giữ nguyên trong named volume.

Workflow này cũng là nền tảng cho phần Docker Compose tiếp theo. Bước chuẩn bị model và backend sẽ được tổ chức thành các service riêng, nhưng vẫn sử dụng chung một model volume.

V. Vận hành ứng dụng bằng Docker Compose

Ở Mục IV, ta đã tự chạy từng container bằng các lệnh Docker riêng. Khi frontend cũng được đưa vào container, số thành phần cần quản lý tăng lên, kéo theo nhiều volume, biến môi trường, port và quan hệ khởi động cần được theo dõi đồng thời.

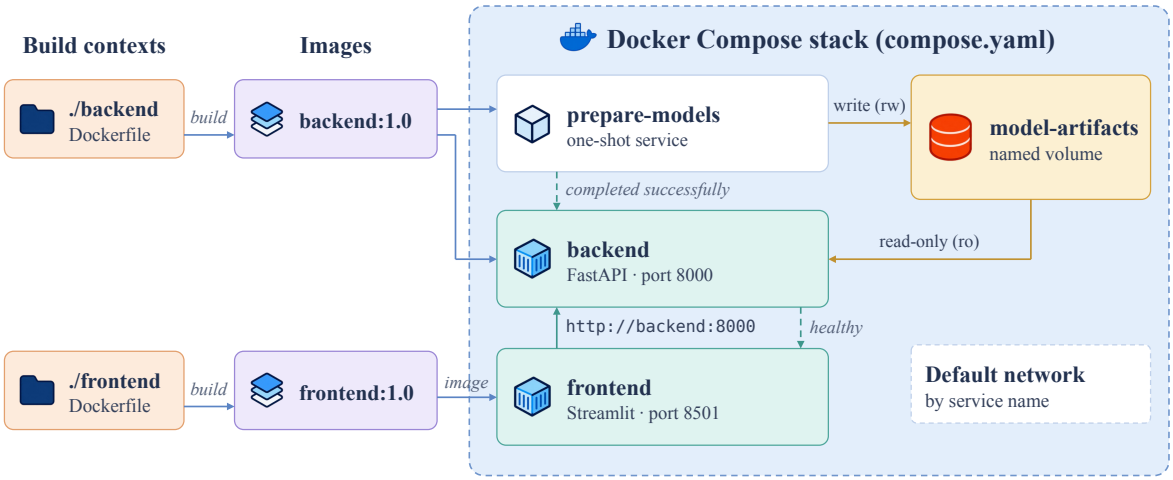
Docker Compose

Là công cụ dùng để định nghĩa và chạy ứng dụng gồm nhiều container thông qua một file cấu hình, thường là `compose.yaml`.

Thay vì duy trì nhiều lệnh `docker run` riêng, project sử dụng Compose để quản lý ba service từ hai image và hai build context:

- `prepare-models` sử dụng image của backend để chuẩn bị model artifacts rồi kết thúc.
- `backend` chạy FastAPI và sử dụng các artifacts đã được chuẩn bị để thực hiện inference.
- `frontend` chạy Streamlit và gửi request tới backend.

Service chuẩn bị model và backend cùng sử dụng volume `model-artifacts`, trong đó `prepare-models` cần quyền đọc và ghi để tạo artifacts, còn backend chỉ cần đọc các file đã được chuẩn bị. Frontend không truy cập model trực tiếp mà gọi API của backend qua `http://backend:8000` trong default network do Compose tạo ra.



Hình 12: Tổng quan mối quan hệ giữa build contexts, images, services, volume dùng chung cho model artifacts và kết nối giữa frontend với backend.

V.1. Tổ chức các thành phần của ứng dụng

Khi đọc file Compose, cần phân biệt **image**, **service** và **container**. Image là nền tảng để tạo container, còn service mô tả một thành phần của ứng dụng sẽ được Compose chạy như thế nào.

Service

Là một thành phần của ứng dụng được mô tả trong Compose, bao gồm image và cách container của thành phần đó được chạy, chẳng hạn command, biến môi trường, volume và restart policy.

Trong project này, ba service được tổ chức từ hai image và hai build context:

compose.yaml: ba service từ hai image

```yaml
services:
  prepare-models:
    image: backend:1.0
    build: ./backend
    command: ["python", "scripts/prepare_models.py"]

  backend:
    image: backend:1.0
    build: ./backend

  frontend:
    image: frontend:1.0
    build: ./frontend
```

Hai service phía backend cùng sử dụng image `backend:1.0`, được build từ thư mục `./backend`, nhưng khi chạy chúng vẫn tạo ra hai container riêng. Backend giữ command mặc định trong image để khởi động Uvicorn, còn `prepare-models` ghi đè command bằng `python scripts/prepare_-`

`models.py` để thực hiện bước chuẩn bị model. Phía frontend sử dụng image `frontend:1.0`, được build từ `./frontend`, và image này khởi động Streamlit trên port `8501`.

Backend image còn đặt sẵn `MODEL_DIR=/models` bằng `ENV`. Vì cả container backend và container chuẩn bị model đều được tạo từ image này, chúng cùng kế thừa giá trị mặc định đó. Script `prepare_models.py` vì vậy biết nơi cần ghi artifacts dù service `prepare-models` không khai báo thêm `MODEL_DIR` trong `environment`.

Mỗi lần được thực thi, `prepare-models` chuẩn bị model artifacts rồi kết thúc thay vì tiếp tục chờ request.

One-shot service

Là service thực hiện một task rồi kết thúc, thay vì duy trì process để tiếp tục phục vụ request.

Sự khác nhau về thời gian hoạt động cũng được thể hiện qua restart policy:

- `prepare-models` dùng `restart: "no"`, nên container không tự khởi động lại sau khi process kết thúc.
- `backend` và `frontend` dùng `restart: unless-stopped`, phù hợp với các service cần tiếp tục hoạt động trong quá trình sử dụng ứng dụng.

One-shot chỉ mô tả hành vi của service trong một lần thực thi, không có nghĩa bước chuẩn bị model chỉ xảy ra một lần trong suốt vòng đời của volume. Nếu `prepare-models` được thực thi lại, script có thể gặp artifacts đã tồn tại nhưng vẫn tiếp tục các bước chuẩn bị tương ứng. Với các model từ Hugging Face, script vẫn gọi `snapshot_download` trong bước chuẩn bị, vì vậy việc `model-artifacts` đã có dữ liệu không có nghĩa `prepare-models` chắc chắn sẽ chạy offline.

Chia sẻ model artifacts qua volume

Model artifacts không được package trực tiếp vào backend image. Compose mount volume `model-artifacts` vào service chuẩn bị model và backend, còn script `prepare_models.py` là thành phần ghi dữ liệu vào volume này:

compose.yaml: volume mount của prepare-models và backend

```
prepare-models:
  volumes:
    - model-artifacts:/models

backend:
  volumes:
    - model-artifacts:/models:ro
```

Hai service cùng truy cập dữ liệu tại `/models` nhưng với quyền khác nhau:

- Service chuẩn bị model có quyền đọc và ghi để tạo hoặc cập nhật artifacts.
- Backend mount cùng volume ở chế độ `ro`, tức chỉ đọc, vì service này chỉ cần sử dụng các file đã được chuẩn bị để chạy inference.

Quyền `ro` chỉ áp dụng cho mount `/models` của backend, không làm toàn bộ filesystem trong container trở thành read-only. Việc giới hạn quyền đối với model artifacts theo đúng nhu cầu của từng service là một ví dụ của nguyên tắc **least privilege**.

Named volume được khai báo ở cuối file với tên cố định:

compose.yaml: khai báo named volume

```
volumes:
  model-artifacts:
    name: model-artifacts
```

Thuộc tính `name` yêu cầu Docker sử dụng đúng tên `model-artifacts` thay vì thêm Compose project name vào phía trước, nhờ đó workflow Compose có thể tiếp tục sử dụng volume đã được tạo và kiểm tra ở Mục IV. Việc đặt tên cố định không biến volume thành external volume. `docker compose down` vẫn giữ named volume, còn `docker compose down -v` sẽ xóa volume cùng model artifacts bên trong.

Cấu hình khi container chạy

Ngoài image và volume, backend còn cần một số giá trị được sử dụng khi container chạy. Các giá trị này được khai báo qua `environment`:

compose.yaml: environment của backend

```
environment:
  MODEL_DIR: /models
  INFERENCE_DEVICE: cpu
  MOCK_MODE: "0"
  HF_HUB_OFFLINE: "1"
```

Mỗi biến đảm nhiệm một vai trò riêng:

- `MODEL_DIR` xác định nơi backend tìm model artifacts.
- `INFERENCE_DEVICE` chọn thiết bị dùng cho inference.
- `MOCK_MODE` kiểm soát việc chạy model thật hay mock.
- `HF_HUB_OFFLINE` yêu cầu backend runtime sử dụng dữ liệu đã có sẵn thay vì truy cập Hugging Face Hub.

Compose khai báo lại `MODEL_DIR=/models` trong `environment` để cấu hình runtime của backend hiển thị rõ ngay trong định nghĩa service. Riêng `HF_HUB_OFFLINE=1` chỉ được truyền cho backend, nên cấu hình này không làm toàn bộ workflow Compose trở thành offline.

Phía frontend chỉ cần biết địa chỉ của API để gửi request, nên cấu hình ngắn hơn:

compose.yaml: environment của frontend

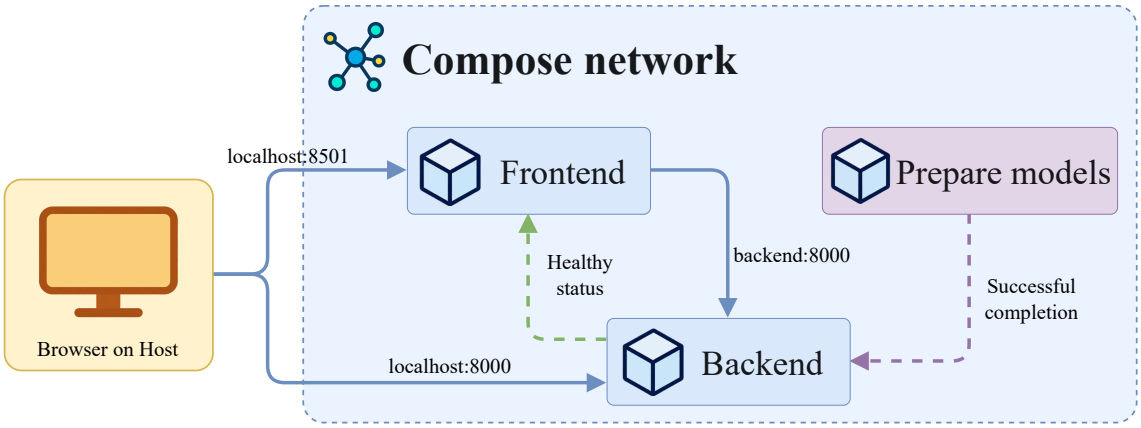
```
environment:
  BACKEND_URL: http://backend:8000
```

Địa chỉ trên sử dụng tên service `backend` thay cho một IP cố định. Cách Compose giúp frontend tìm thấy backend bằng service name sẽ được giải thích ở Mục V.2.

Tìm hiểu thêm. Khi số lượng biến môi trường tăng hoặc cần thay đổi giữa development, testing và production, Compose có thể lấy giá trị cho variable interpolation từ `.env`, còn `env_file` giúp load một nhóm biến vào môi trường của container. Với token, password hoặc API key, nên dùng cơ chế quản lý secrets phù hợp thay vì ghi trực tiếp các giá trị nhạy cảm vào file cấu hình.

V.2. Kết nối các service và xác định thứ tự khởi động

Sau khi các service đã được tổ chức trong Compose, ứng dụng cần xử lý hai mối quan hệ khác nhau. Khi chạy, frontend phải biết cách gửi request tới backend. Trong quá trình khởi động, backend chỉ nên khởi động sau khi `prepare-models` hoàn tất thành công, còn frontend cần chờ backend phản hồi được `/health`.



Hình 13: Giao tiếp giữa các service trong mạng Compose và các điều kiện về thứ tự khởi động.

Trong hình, mũi tên liền biểu diễn HTTP request, còn mũi tên nét đứt biểu diễn dependency khi khởi động. Browser/host nằm ngoài Compose network, trong khi frontend và backend được kết nối vào default network do Compose tạo ra.

Giao tiếp giữa các service

File `compose.yaml` không khai báo network riêng, nên Compose tạo một default network và kết nối các service của ứng dụng vào đó. Trong network này, các service có thể được truy cập bằng tên đã khai báo trong Compose thay vì phải cố định địa chỉ IP của container.

Service discovery

Là cơ chế giúp một service tìm và kết nối tới service khác bằng một tên ổn định thay vì phụ thuộc vào địa chỉ IP của container.

Frontend sử dụng cơ chế này thông qua `BACKEND_URL`:

compose.yaml: BACKEND_URL và port mapping

```yaml
frontend:
  environment:
    BACKEND_URL: http://backend:8000
  ports:
    - "8501:8501"

backend:
  ports:
    - "8000:8000"
```

Trong địa chỉ `http://backend:8000`, `backend` chính là service name được dùng như hostname trong default network của Compose. Frontend vì vậy có thể tìm backend bằng tên này mà không cần biết IP hiện tại của backend container.

Cần phân biệt địa chỉ dùng từ máy host với địa chỉ dùng giữa các container:

- Browser truy cập frontend qua `http://localhost:8501`.
- Browser có thể truy cập trực tiếp backend qua `http://localhost:8000`.
- Frontend container gọi backend qua `http://backend:8000`.

Với mapping `"8000:8000"`, port bên trái được publish ra máy host, còn port bên phải là port FastAPI lắng nghe trong backend container. Do frontend và backend cùng nằm trên Compose network, frontend có thể gọi trực tiếp `backend:8000` mà không cần đi qua host port. Nếu frontend sử dụng `localhost:8000`, `localhost` sẽ trỏ về chính frontend container chứ không phải backend.

Default network đã đủ cho cấu trúc hiện tại. Khi ứng dụng lớn hơn và cần giới hạn service nào được phép giao tiếp trực tiếp với nhau, Compose cho phép chia các service sang nhiều custom network. Chẳng hạn, trong kiến trúc gồm frontend, API và database, frontend có thể chỉ kết nối với API, còn database chỉ cho API truy cập.

Thứ tự khởi động và trạng thái service

Backend cần chờ bước chuẩn bị model hoàn tất thành công, còn frontend cần chờ backend phản hồi được `/health`. Hai điều kiện này được mô tả bằng `depends_on`:

compose.yaml: depends_on và healthcheck

```yaml
backend:
  depends_on:
    prepare-models:
      condition: service_completed_successfully
```

```
    healthcheck:
      test:
        [
          "CMD",
          "python",
          "-c",
          "import urllib.request; urllib.request.urlopen('http://127.0.0.1:8000/health',
                                                         timeout=3)"
        ]
      interval: 10s
      timeout: 5s
      retries: 5
      start_period: 30s

  frontend:
    depends_on:
      backend:
        condition: service_healthy
```

Hai dependency sử dụng hai điều kiện khác nhau. Backend chờ service chuẩn bị model kết thúc thành công thông qua `service_completed_successfully`, phù hợp với cách hoạt động của one-shot service. Frontend phụ thuộc vào một service chạy lâu dài, nên dùng `service_healthy` để chờ backend vượt qua healthcheck rồi mới khởi động frontend.

Trình tự startup gồm:

- `prepare-models` hoàn thành thành công.
- Backend được khởi động.
- Backend vượt qua healthcheck và chuyển sang trạng thái `healthy`.
- Frontend được khởi động.

Docker xác định trạng thái của backend thông qua healthcheck được khai báo ngay trong service.

Healthcheck

Là phép kiểm tra được Docker thực hiện định kỳ để xác định container đang ở trạng thái `healthy` hay `unhealthy` dựa trên một command được cấu hình trước.

Trong project này, healthcheck dùng Python standard library `urllib.request` để gọi `http://127.0.0.1:8000/health`. Lệnh chạy bên trong backend container, nên `127.0.0.1` trỏ tới chính FastAPI server trong container đó. Trường hợp này khác với frontend vì frontend nằm trong một container riêng và phải tìm backend qua `backend:8000`.

Các parameter còn lại kiểm soát cách healthcheck được thực hiện:

- `interval: 10s` đặt khoảng thời gian giữa các lần kiểm tra.
- `timeout: 5s` giới hạn thời gian tối đa cho mỗi lần kiểm tra.

- `retries: 5` xác định năm lần kiểm tra thất bại liên tiếp trước khi container bị đánh dấu `unhealthy`.
- `start_period: 30s` dành một khoảng thời gian cho backend khởi động, trong đó các lần kiểm tra thất bại ban đầu chưa được tính vào giới hạn retry.

`start_period` không có nghĩa Docker đợi đủ 30 giây rồi mới bắt đầu healthcheck. Các phép kiểm tra vẫn có thể chạy trong khoảng này, nhưng những lần thất bại ban đầu được xử lý như một phần của giai đoạn startup.

Khi quan sát backend, cần phân biệt ba mức kiểm tra:

- **Running** cho biết process trong container đang chạy.
- **Healthy** cho biết healthcheck đang thành công, trong project này là endpoint `/health` phản hồi theo yêu cầu.
- **Inference thành công** xác nhận backend thực sự load được model artifacts và xử lý được một request model thật.

Healthcheck không thay thế cho smoke test inference. Trong các hệ thống lớn hơn, việc kiểm tra process còn hoạt động và việc service đã thực sự sẵn sàng phục vụ thường được tách thành **liveness** và **readiness**. Với project này, `/health` chỉ là tín hiệu cho biết API đã phản hồi, dùng để điều phối việc khởi động frontend, không xác nhận model đã load hoặc inference đã hoạt động thành công.

Điều kiện trong `depends_on` cũng chỉ được dùng để phối hợp quá trình khởi động. Sau khi frontend đã chạy, nếu backend chuyển sang trạng thái `unhealthy`, Compose không tự động dừng hoặc restart frontend chỉ vì dependency health không còn đạt điều kiện.

V.3. Kiểm tra cấu hình và vận hành ứng dụng

Sau khi các service, networking và quan hệ phụ thuộc đã được thiết lập, bước cuối cùng là vận hành toàn bộ ứng dụng bằng Docker Compose CLI. Workflow trong phần này đi theo một luồng duy nhất: kiểm tra cấu hình, build image, khởi động stack ở background, theo dõi log, kiểm tra trạng thái, xác minh ứng dụng rồi dừng Compose.

Kiểm tra cấu hình và khởi động ứng dụng

Trước khi tạo container, nên kiểm tra cấu hình mà Compose thực sự sẽ sử dụng bằng `docker compose config`. Lệnh này giúp phát hiện lỗi cấu hình và cho phép rà lại các thành phần như service, image, build context, biến môi trường, port mapping, volume mount và quan hệ phụ thuộc trước khi ứng dụng được khởi động.

PowerShell: kiểm tra cấu hình Compose

```
docker compose config
```

Khi cấu hình đã hợp lệ, build các image được khai báo trong Compose bằng:

PowerShell: build các image bằng Compose

```
docker compose build
```

Thay vì phải chạy từng lệnh `docker build` riêng cho backend và frontend, Compose đọc trực tiếp các build context đã được khai báo trong `compose.yaml` và thực hiện quá trình build tương ứng. Sau đó, khởi động toàn bộ stack ở background bằng:

PowerShell: khởi động stack ở background

```
docker compose up -d
```

Detached mode

Là chế độ chạy container ở background, nhờ đó terminal được trả lại cho người dùng trong khi các service tiếp tục hoạt động.

Để xem lại log của các service và tiếp tục theo dõi log mới, dùng:

PowerShell: theo dõi log của các service

```
docker compose logs -f
```

Lệnh này hiển thị cả những log đã được ghi trước thời điểm chạy command rồi tiếp tục theo dõi các log mới. Vì vậy, ngay cả khi `prepare-models` đã hoàn thành trước đó, output của bước chuẩn bị model vẫn có thể được xem lại. Khi đã quan sát đủ thông tin, nhấn `Ctrl+C` để dừng việc theo dõi log. Thao tác này chỉ thoát khỏi `logs -f`, các container vẫn tiếp tục chạy ở background.

Quan sát trạng thái và thử các chức năng

Tiếp theo, kiểm tra trạng thái của toàn bộ stack bằng:

PowerShell: kiểm tra trạng thái stack

```
docker compose ps -a
```

Tùy chọn `-a` giúp hiển thị cả container đang chạy lẫn container đã kết thúc. Điều này cần thiết với `prepare-models` vì đây là one-shot service. Kết quả mong đợi là `prepare-models` ở trạng thái `Exited (0)`, backend đang chạy và đạt trạng thái `healthy`, còn frontend tiếp tục chạy để phục vụ UI.

```
PS D:\AIO\Project\docker-for-ai-applications> docker compose logs -f
backend-1          | INFO:     127.0.0.1:45074 - "GET /health HTTP/1.1" 200 OK
backend-1          | INFO:     127.0.0.1:56092 - "GET /health HTTP/1.1" 200 OK

PS D:\AIO\Project\docker-for-ai-applications> docker compose ps -a
NAME                                          IMAGE          COMMAND                  SERVICE
        CREATED         STATUS                     PORTS
docker-for-ai-applications-backend-1          backend:1.0    "uvicorn main:app --…"   backend
        2 minutes ago   Up 2 minutes (healthy)     0.0.0.0:8000->8000/tcp, [::]:8000->8000/tcp
docker-for-ai-applications-frontend-1         frontend:1.0   "streamlit run app.p…"   frontend
        2 minutes ago   Up 2 minutes               0.0.0.0:8501->8501/tcp, [::]:8501->8501/tcp
docker-for-ai-applications-prepare-models-1   backend:1.0    "python scripts/prep…"   prepare-m
odels   2 minutes ago   Exited (0) 2 minutes ago
```

Hình 14: Trạng thái các service sau khi khởi động bằng Docker Compose.

Trạng thái container mới chỉ cho biết các thành phần đã khởi động theo đúng lifecycle. Để kiểm tra backend thực sự phản hồi request, trước tiên gọi endpoint `/health` từ máy host:

PowerShell: kiểm tra /health của backend

```
Invoke-RestMethod http://localhost:8000/health
```

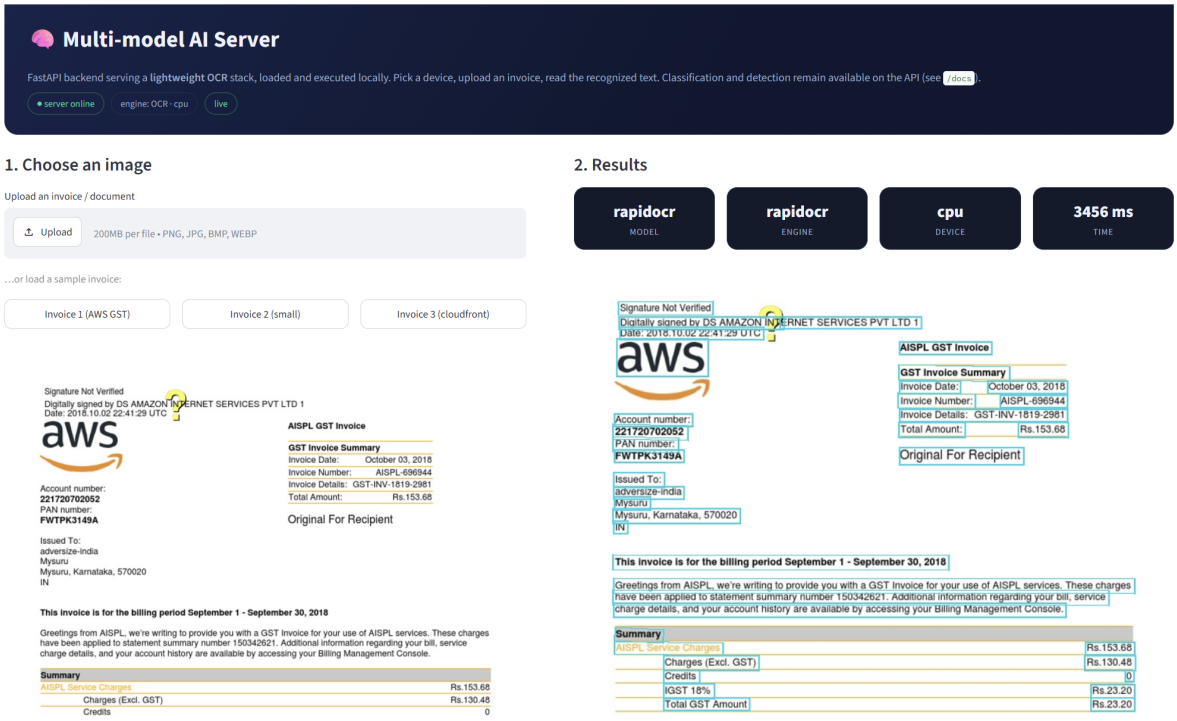
Sau đó dùng lại Iris classification request ở Mục IV để kiểm tra backend có thể đọc model artifact từ volume và thực hiện inference thật trên CPU:

PowerShell: gửi lại request Iris classification

```
$body = @{
  sepal_length = 5.1
  sepal_width = 3.5
  petal_length = 1.4
  petal_width = 0.2
} | ConvertTo-Json

Invoke-RestMethod `
  -Uri http://localhost:8000/classification/predict `
  -Method Post `
  -ContentType "application/json" `
  -Body $body
```

Nếu request trả về prediction, backend không chỉ vượt qua healthcheck mà còn có thể load model artifact từ `model-artifacts` và thực hiện inference. Bước kiểm tra cuối cùng là mở frontend tại `http://localhost:8501`, gửi một request OCR từ UI và xác nhận kết quả được trả về. Khi đó, luồng `frontend → backend → inference → result` đã được kiểm tra từ đầu đến cuối.



Hình 15: Kết quả OCR khi kiểm tra luồng frontend–backend trên ứng dụng.

Dừng ứng dụng

Sau khi hoàn tất các bước kiểm tra, dừng ứng dụng và xóa các container cùng network do Compose quản lý bằng:

PowerShell: dừng stack, xóa container và network

```
docker compose down
```

Named volume `model-artifacts` vẫn được giữ lại, vì vậy model artifacts không bị xóa cùng container và có thể tiếp tục được sử dụng ở những lần chạy sau.

Lưu ý: Không thêm `-v` nếu muốn giữ model artifacts. Lệnh dưới đây sẽ xóa cả named volume được khai báo trong Compose:

PowerShell: down kèm xóa named volume

```
docker compose down -v
```

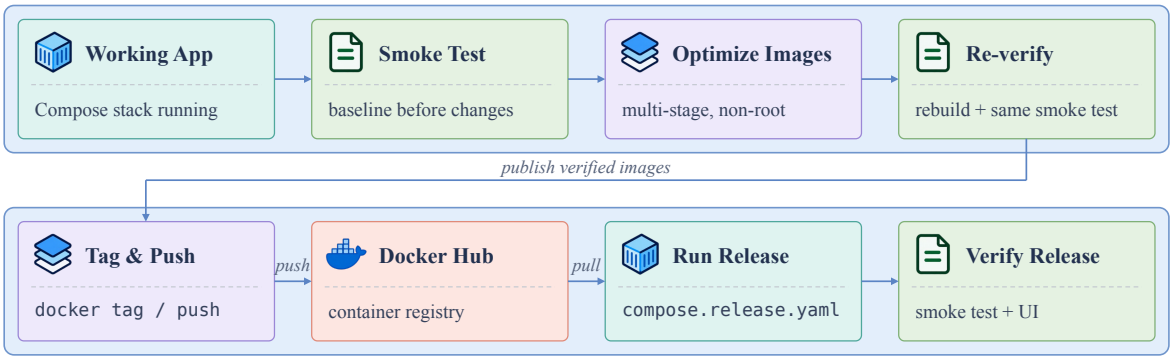
Với project này, điều đó đồng nghĩa dữ liệu trong `model-artifacts` cũng bị xóa. Đây không phải là bước tiếp theo trong workflow thực hành mà chỉ dùng khi thực sự muốn loại bỏ cả dữ liệu model.

Tìm hiểu thêm: Workflow trong tutorial sử dụng một `compose.yaml` và khởi động toàn bộ service cần thiết cho ứng dụng. Khi project phát triển, Compose còn cung cấp các cơ chế để điều chỉnh cách vận hành mà không cần đưa mọi trường hợp vào cùng một cấu hình. Nhiều Compose file có thể được kết hợp để giữ phần cấu hình chung ở file chính rồi bổ sung hoặc ghi đè những

phần dành riêng cho development, testing hay production. Các service chỉ cần trong một số tình huống, chẳng hạn debugger hoặc monitoring tool, có thể được đặt trong **Profiles** để chỉ được bật khi cần. Trong quá trình phát triển source code, **Compose Watch** có thể theo dõi thay đổi và tự đồng bộ hoặc rebuild service tương ứng, giúp rút ngắn vòng lặp sửa code và chạy lại container.

VI. Tối ưu và publish Docker image

Đến thời điểm này, application đã được containerize hoàn chỉnh và có thể chạy qua Docker Compose. Frontend cũng cho phép người dùng trực tiếp thử các chức năng inference, vì vậy vấn đề tiếp theo không còn là làm sao để application chạy được, mà là các Docker image hiện tại đã đủ gọn, an toàn và thuận tiện để chia sẻ hay chưa. Đây là lúc chuyển từ một image chủ yếu được tạo để application hoạt động đúng sang một image được chuẩn bị kỹ hơn cho việc sử dụng và phát hành.



Hình 16: Tổng quan quá trình từ application đang hoạt động đến Docker image được tối ưu, kiểm tra lại và publish.

Để thực hiện bước chuyển này, cần xem xét backend và frontend image có đang mang theo những thành phần không còn cần thiết khi application chạy hay không, container có cần sử dụng quyền `root` hay không, và image cần được quản lý như thế nào để có thể xác định đúng version khi sử dụng lại. Sau khi được tối ưu, application sẽ được kiểm tra lại để bảo đảm các thay đổi không ảnh hưởng đến chức năng hiện có, trước khi các image được tag version và publish để có thể pull về và chạy trên môi trường khác.

VI.1. Kiểm tra các chức năng chính của ứng dụng

Ở các mục trước, frontend đã cho phép người dùng trực tiếp thử các chức năng của application. Tuy nhiên, khi bắt đầu thay đổi Dockerfile để tối ưu image, việc lặp lại toàn bộ thao tác thủ công qua giao diện sau mỗi lần build sẽ không thuận tiện. Một smoke test giúp tạo ra một cách kiểm tra ngắn gọn và nhất quán để xác nhận các chức năng chính vẫn hoạt động sau những thay đổi đó.

Smoke test

Là một kiểm tra nhanh nhằm xác nhận những chức năng quan trọng của application vẫn có thể hoạt động trước khi thực hiện các kiểm tra sâu hơn.

Trong project này, `scripts/smoke_test.py` được đặt ở cấp repo và chạy từ máy host sau khi Docker Compose stack đã khởi động. Script lần lượt kiểm tra trạng thái backend, Iris classification, RapidOCR và object detection bằng các request thực tế. Classification sử dụng trực tiếp bốn số đo của hoa Iris, trong khi OCR và detection dùng một ảnh PPM nhỏ được tạo ngay trong memory, nhờ đó không cần thêm file ảnh test hay dependency riêng.

PowerShell: chạy smoke test

```
# Xem toàn bộ scripts/smoke_test.py tại Phụ lục ...
python scripts\smoke_test.py
```

Mỗi request phải hoàn tất thành công và trả về response phù hợp với model được cấu hình. Bước kiểm tra '/health' chỉ xác nhận API đang phản hồi, còn việc model hoạt động được xác nhận bởi các request classification, OCR và detection thành công. Nếu một request gặp lỗi hoặc một điều kiện kiểm tra không thỏa mãn, script sẽ dừng và báo lỗi.

Một lần chạy thành công cho kết quả:

Output của scripts/smoke_test.py

```
[ok] health
[ok] classification
[ok] ocr
[ok] detection
Smoke test passed.
```

Khi smoke test chạy trọn vẹn, ta có một mốc kiểm tra ổn định cho trạng thái hiện tại của application. Các request đã đi qua những đường xử lý chính và được phục vụ bằng model thật. Với OCR và object detection, ảnh được tạo trong script chỉ cần đủ hợp lệ để kích hoạt luồng xử lý, nên kết quả này phù hợp để kiểm tra application vẫn vận hành như mong đợi sau khi Docker image được thay đổi.

VI.2. Tối ưu Docker image

Smoke test ở Mục VI.1 đã cho ta một mốc đối chiếu application trước và sau khi thay đổi Dockerfile. Ở version ban đầu, backend và frontend đều cài dependencies rồi chạy application trong cùng một image. Khi application đã hoạt động ổn định, quá trình build có thể được tổ chức lại để tách phần chuẩn bị dependencies khỏi môi trường thực sự dùng để chạy.

Multi-stage build

Cho phép một Dockerfile sử dụng nhiều stage, trong đó image cuối chỉ nhận những thành phần được chọn từ các stage trước.

Các đoạn Dockerfile dưới đây chỉ giữ những instruction liên quan trực tiếp đến phần tối ưu đang xét, không phải nội dung đầy đủ để build image.

Với backend, builder tạo virtual environment tại `/opt/venv` và cài Python dependencies vào đó. PyTorch và torchvision cũng được lấy từ CPU index vì application chỉ chạy inference trên CPU. Runtime stage sau đó nhận virtual environment đã được chuẩn bị sẵn.

backend/Dockerfile: multi-stage build với virtual environment

```
FROM python:3.11-slim AS builder

ENV VIRTUAL_ENV=/opt/venv \
    PATH="/opt/venv/bin:$PATH"

RUN python -m venv "$VIRTUAL_ENV"

COPY requirements.txt .

RUN pip install --no-cache-dir \
        torch==2.14.1 \
        torchvision==0.29.1 \
        --index-url https://download.pytorch.org/whl/cpu \
    && pip install --no-cache-dir -r requirements.txt

FROM python:3.11-slim AS runtime

COPY --from=builder /opt/venv /opt/venv

# Xem Dockerfile đầy đủ tại Phụ lục.
```

Frontend cũng sử dụng hai stage, nhưng builder cài dependencies vào `/install` rồi copy chúng sang `/usr/local` của runtime.

frontend/Dockerfile: multi-stage build với --prefix

```
FROM python:3.11-slim AS builder

COPY requirements.txt .
RUN pip install --no-cache-dir --prefix=/install -r requirements.txt

FROM python:3.11-slim AS runtime

COPY --from=builder /install /usr/local

# Xem Dockerfile đầy đủ tại Phụ lục.
```

Frontend từng được thử với virtual environment tương tự backend, nhưng image thu được khoảng 842 MB. Khi chuyển sang `--prefix`, kích thước giảm xuống khoảng 803 MB. Trường hợp này cho thấy multi-stage build không đồng nghĩa với việc image tự động nhỏ hơn; kết quả vẫn cần được build và đo trên chính application.

Sau khi cập nhật Dockerfile, hai image được build lại và kiểm tra kích thước:

PowerShell: build lại và kiểm tra kích thước image

```
docker compose build
docker image inspect backend:1.0 frontend:1.0 --format '{{index .RepoTags 0}} {{.Size
                                                    }}'
```

`docker image inspect` trả kích thước theo byte. Riêng backend còn có thể kiểm tra phần virtual environment bằng:

PowerShell: đo kích thước /opt/venv

```
docker run --rm --entrypoint du backend:1.0 -sb /opt/venv
```

Trong lần đo của project, `/opt/venv` chiếm khoảng 1,81 GB sau khi quy đổi từ byte, cho thấy phần lớn thư viện inference vẫn phải được giữ lại trong runtime image. Các số đo tổng thể được làm tròn theo MB và GB thập phân như sau. Với backend, mức giảm còn chịu ảnh hưởng lớn từ việc chuyển PyTorch kèm CUDA sang bản dành cho CPU, nên không thể quy toàn bộ chênh lệch cho multi-stage build.

Image	Trước thay đổi	Sau thay đổi
`backend:1.0`	∼10,36 GB	∼2,81 GB
`frontend:1.0`	∼820 MB	∼803 MB

Một điểm khác cần xem xét khi hoàn thiện image là user được dùng để chạy process trong container. Nếu Dockerfile không chuyển sang user khác, process thường chạy với `root`, trong khi backend và frontend của project không cần mức quyền này khi phục vụ request. Hai runtime image vì vậy tạo user `app` với UID/GID `10001` và sử dụng `USER app` cho process chính. `prepare-models` là ngoại lệ hiện tại vì cần ghi model artifacts vào volume, nên service này được Compose cấu hình với `user: "0:0"`.

Sau khi image đã được build, hãy khởi động stack và kiểm tra trạng thái trước khi chạy lại các request inference:

PowerShell: khởi động stack và chạy smoke test

```
docker compose up -d
docker compose ps -a
python scripts\smoke_test.py
```

Ở lần kiểm tra frontend gần nhất, container đã chạy bằng UID `10001`, trả HTTP `200` và vẫn kết nối được tới `/health` của backend. Lệnh smoke test phía trên giúp người đọc kiểm tra lại các đường inference trên image đã tối ưu. Chỉ khi smoke test chạy thành công mới có thể kết luận các inference path vẫn hoạt động sau thay đổi.

VI.3. Đưa image lên registry và chạy bản release

Sau khi các image đã được tối ưu và kiểm tra, bước tiếp theo là đưa chúng lên một container registry để có thể được lưu trữ và sử dụng lại mà không cần build từ source. Trong project này, Docker Hub được sử dụng cho backend và frontend image.

Container registry

Là service dùng để lưu trữ và phân phối container image.

Trước khi publish, namespace của Docker Hub và version được khai báo thành biến để các lệnh phía sau có thể sử dụng lại. Người đọc chỉ cần thay `<docker-hub-username>` bằng namespace của tài khoản hoặc tổ chức tương ứng.

PowerShell: tag và push image lên Docker Hub

```
$env:REGISTRY_NAMESPACE = "<docker-hub-username>"
$env:IMAGE_TAG = "1.0"

$backendImage = "$($env:REGISTRY_NAMESPACE)/multi-model-ai-backend:$($env:IMAGE_TAG)"
$frontendImage = "$($env:REGISTRY_NAMESPACE)/multi-model-ai-frontend:$($env:IMAGE_TAG)"

docker login

docker tag backend:1.0 $backendImage
docker tag frontend:1.0 $frontendImage

docker push $backendImage
docker push $frontendImage
```

Sau khi push hoàn tất, Docker Hub sẽ chứa version `1.0` của hai image và trả về digest tương ứng. Tag và digest ở đây tiếp tục sử dụng cơ chế nhận diện image đã được giới thiệu ở Mục III.3, nên không cần tạo lại image khi chỉ thay đổi cách tham chiếu đến nó.

Để chạy application từ registry, project sử dụng `compose.release.yaml`. File này tham chiếu tới image thông qua `REGISTRY_NAMESPACE` và `IMAGE_TAG` thay vì chứa `build:` như cấu hình development. Trước khi dừng stack đang chạy, có thể để Compose resolve toàn bộ cấu hình và kiểm tra lại các image reference:

PowerShell: kiểm tra compose.release.yaml

```
docker compose -p ai-release -f compose.release.yaml config
```

Trong kết quả, các service cần trỏ tới đúng backend hoặc frontend image trên registry và không còn `build:`. `prepare-models` và backend cùng sử dụng backend image, còn frontend sử dụng image riêng.

Sau khi cấu hình release đã đúng, development stack được dừng để giải phóng các port đang sử dụng. Bản release được chạy với project name `ai-release`, sau đó các image được pull từ registry trước khi container được khởi động.

PowerShell: chạy bản release từ registry

```
docker compose down

docker compose -p ai-release -f compose.release.yaml pull --policy always
docker compose -p ai-release -f compose.release.yaml up -d
docker compose -p ai-release -f compose.release.yaml ps -a
```

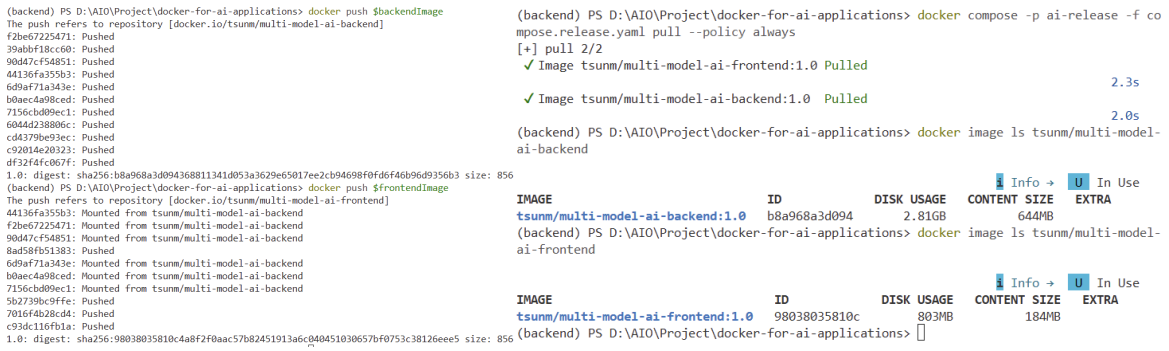
Trong `compose.release.yaml`, volume `model-artifacts` không được đặt `name:` cố định, vì vậy Compose đặt tên resource dựa trên project name. Ở lần chạy đầu của `ai-release`, điều này tạo một model volume tách khỏi volume của development stack. Nếu project `ai-release` đã được chạy trước đó và volume vẫn còn, Compose có thể sử dụng lại volume đó. Khi volume chưa có model artifacts, `prepare-models` sẽ chuẩn bị chúng trước khi backend sẵn sàng.

Khi stack đã ổn định, trạng thái mong đợi là `prepare-models` kết thúc thành công, backend ở trạng thái `healthy` và frontend tiếp tục chạy. Smoke test ở Mục VI.1 được dùng lại để kiểm tra các đường xử lý chính:

PowerShell: chạy smoke test

```
python scripts\smoke_test.py
```

Có thể mở thêm `http://localhost:8501` và thử một request từ UI để kiểm tra luồng sử dụng thực tế.



Hình 17: Kết quả publish backend và frontend image lên Docker Hub, pull lại version `1.0` để chạy bản release, sau đó kiểm tra trạng thái các service và smoke test.

Trong phần thực hành này, các image được push lên Docker Hub rồi pull lại trên cùng máy, nhưng được chạy bằng Compose project và model volume riêng với development stack. Workflow

tương tự có thể được sử dụng trên một máy khác khi môi trường đó có file cấu hình release, quyền truy cập image và các điều kiện cần thiết để chuẩn bị model artifacts.

Tìm hiểu thêm: Sau khi đã có một workflow tương đối hoàn chỉnh từ build, kiểm chứng đến publish image, quá trình phát triển hằng ngày vẫn có thể được cải thiện thêm bằng **Dev Containers**, cách tách **development image** khỏi **runtime image**, hoặc các **project template** để tái sử dụng cấu trúc Docker đã ổn định cho những ứng dụng sau.

Phụ lục

1. **Solution:** Các file code cài đặt hoàn chỉnh và phần trả lời nội dung trắc nghiệm có thể được tải tại đây.
2. **Q&A:** Mọi người có thể đặt thêm câu hỏi về nội dung bài đọc trong group Facebook hỏi đáp tại đây. Tất cả câu hỏi sẽ được trả lời trong vòng tối đa 4 tiếng.

